Auteur : Sébastien GELDREICH

Date : 31/07/2020

Trehinos EMS (« THOR ») – Document technique

Entreprise Management System

Trehinos EMS est un framework de système de gestion d'entreprise. Il permet de développer rapidement le CRM (logiciel de relation client) et l'ERP (logiciel de gestion) de l'entreprise dans une application multi-utilisateurs extensive.

Sommaire

SOMMAIRE **2**

TECHNOLOGIES **4**

SERVEUR 4
CLIENT 4

DISTRIBUTION **5**

LICENCE UTILISATEUR DE TREHINOS EMS 5
Licence EMS Pro *5*
CIBLE 5
PREREQUIS 5
VERSION ACTUELLE : 0.1 5
INSTALLATION 5
CONFIGURATION 5
COMMENCER A DEVELOPPER LE PACKAGE PRINCIPAL 5
ARCHITECTURE ET DROITS 6

DECOUPAGE FONCTIONNEL **7**

ROLES 7
Rôles spéciaux hiérarchisés *7*
FLUX DE TRAVAIL 7
DEVELOPPEMENT 7
Application *7*
Gestionnaire de version *7*
Packages *8*
Extensions *8*
Doctrine *8*
Actions *8*
CONCEPTION 8
Domaine *8*
Vues, Formulaires, Eléments *8*
Pages *8*
Thèmes *8*
ADMINISTRATION 8
Utilisateurs et rôles *8*
Paramètres *8*
UTILISATEURS 9
Affichage des pages *9*
Barre d'outils 9
Extensions 9
Notifications 9
Raccourcis 9
Profil utilisateur 9
Déconnexion 9

Page d'accueil utilisateur *9*
Communication 9
Organisation 9

CONTRAINTES TECHNIQUES 10

CONTRAINTES DE CODAGE 10
Respect des normes du FIG *10*
Normes propres au framework *10*
Conventions proposées aux développeurs-utilisateurs du framework *10*
GESTIONNAIRE DE VERSION ET BRANCHES 11
Branches principales *11*
Branches de développement *11*
Tags *11*

Technologies

Serveur

- Serveur HTTP (développé sur Apache 2)
- Base de données compatible PDO (développé sur MySQL).
- PHP 7.4+ et les pilotes PHP compatible avec la base de données.
- Composer

Bibliothèques pour navigateur

- JQuery
- Bootstrap 4 (bientôt 5)
- FontAwesome *Pro version 5.4+ nécessaire (**99 $**)*

Distribution

Licence utilisateur de Trehinos EMS

La LICENCE « EMS » utilisateur de Trehinos EMS (ci-après désigné le « PROJET ») est le texte permettant à l'auteur du PROJET d'accorder aux utilisateurs signataires de cette LICENCE le droit d'utiliser le PROJET. Elle doit permettre l'utilisation libre du logiciel, la modification des sources, la redistribution telle qu'elle et gratuitement ou sous la même licence (ou modifiée mais sous un autre nom et sans suppression de mention des auteurs originaux et sous licence y compris commerciale au choix et compatible avec les licences déjà utilisées par le PROJET).

Une copie complète de la licence est distribuée dans toutes les versions du PROJET. Le nom de ce fichier DOIT être « documents/EMS_LICENSE ».

Licence EMS Pro

La Licence EMS Pro peut être contractée par un développeur indépendant ou une entreprise pour permettre une assistance et un retour utilisateur pour l'amélioration du projet.

Cible

EMS cible les développeurs en petites et moyennes entreprises cherchant à développer un ERP et/ou un CRM. Il leur offre un cadre de développement idéal.

Prérequis

- *EMS nécessite une licence commerciale de FontAwesome*,
- Avoir un serveur HTTP, PHP 7.4+, et un SGBD,
- Copier le projet EMS,
- Posséder les droits administrateurs pour l'exécution de « `bin/setup.php` ».
  Ce fichier, au travers d'un assistant, :
  - Installe : les dépendances PHP en exécutant Composer,
  - Installe : les dépendances client (Bootstrap, FontAwesome, JQuery),
  - Installe : le package principal (utilisateurs de base, configuration),
  - et donne des conseils de début,
- Modifications des droits des dossiers.

Version actuelle : 0.1-dev

Un POC est disponible ici : `F:\XamppPHP74\htdocs\ems` sur XAMPP.

Installation et configuration de base

…

Commencer à développer le package principal

…

Architecture et droits

Le virtualhost doit pointer sur « `www/index.php` » et pouvoir accéder aux assets (répertoire `www/`), pas aux autres fichiers.

PHP doit pouvoir écrire dans « `ems/packages/` », « `ems/plugins/` » et « `ems/themes/` ».

Dossier	Description	HR	PR	PW	CR	CW
`bin/`	PHP exécutable en mode console				X	
`app/`	Application : configuration et code propre		X			
`…/res/`	Ressources (fichiers YML)		X			
`…/…/config/`	Configuration (éditable)		X		X	
`…/…/static/`	Données statiques (éditables)		X		X	
`…/…/…/langs/`	Données de langues		X		X	
`…/…/views/`	Vues propres à l'application		X			
`…/src/`	Code propre à l'application – App\		X			
`documents/`	Documents du projet (licence, documentation, changelog)		X			
`ems/`	Code de l'application (YML, PHP) –Ems\		X		X	X
`…/packages/`	Sous-systèmes EMS		X	X		
`…/plugins/`	Extensions		X	X		
`…/themes/`	Fichiers TWIG utilisés par les pages		X	X		
`web/`	Fichiers accessibles au serveur http	X	X			
`thor/`	Framework Thor –Thor\		X			
`templates/`	Templates de base. Utilisés par les themes		X			
`tests/`	Tests unitaires				X	
`var/`	Fichiers « variables » (logs, caches)		X	X	X	X
`vendor/`	Bibliothèques PHP installées par Composer et autochargés.		X		X	

H : http (virtual host), **P** : php, **C** : cli / **R** : read, **W** : write

Découpage fonctionnel

Rôles

Principaux rôles :

- Développeur : configure l'application, design la base de données selon le domaine défini.
- Administrateur : paramètre l'EMS, le domaine et les utilisateurs.
- Concepteur : créée les vues selon le domaine et donne accès aux actions.

Rôles = nom (utilisé dans les droits des pages) le nom du rôle ne doit pas être préfixé de « ROLE_ », il est concaténé automatiquement par EMS.

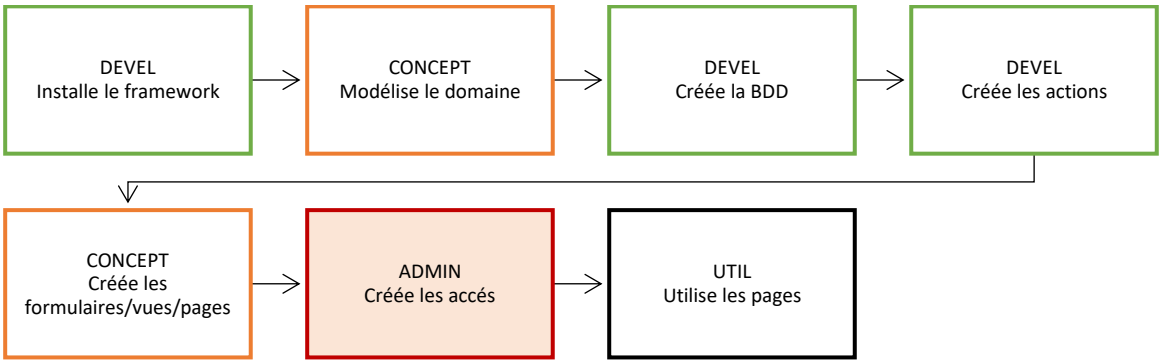
Rôles spéciaux hiérarchisés

`USER < DOMAIN_USER < CONCEPTOR < ADMIN < DEV`

- `USER` = Utilisateur de base, peut se connecter,
- `DOMAIN_USER` = Utilisateur de base du domaine, peut utiliser des pages du domaine,
- `CONCEPTOR` = Concepteur, peut configurer les pages du domaine,
- `ADMIN` = Administrateur, peut gérer les utilisateurs et les rôles,
- `DEV` = Développeur.

Flux de travail

Le flux de travail est dirigé des développeurs vers les utilisateurs. Les développeurs structurent l'application, les concepteurs lient les données techniques au domaine/métier, les administrateurs gèrent les accès et les utilisateurs utilisent les pages de l'application pour en éditer les données.



Développement

Les développeurs sont ceux qui installent le framework, ils peuvent aussi l'étendre, le mettre à jour et ont accès à toutes les interfaces de l'application. Ce sont les seuls à avoir accès au code de l'application. Les actions élémentaires, notamment, sont des bouts de codes (méthodes de classes).

Application

Suspension, mise à jour EMS, mise à jour Application (correspond à un déploiement DEV > PROD), sauvegarde, restauration.

Gestionnaire de version

Disponible pour l'environnement de DEV uniquement. Permet de faire des commit, de visualiser et gérer les branches, versions et distributions.

Packages
Exportation du package principal et installation des packages depuis un dépôt centralisé. Un package correspond à une « sous-application ».

Un package correspond à l'ensemble (configuration, packages, extensions, base de données (entités & données),

Extensions
Installation, mise à jour et configuration des extensions.

Doctrine
Entités, relations et exploration/modification et configuration de la base de données.

Actions
Liens entre URLs et actions (code).

Conception

Domaine
Modèle conceptuel des données et modèle conceptuel des traitements.

Vues, Formulaires, Eléments
Eléments (menu, élément de formulaire, table, modale) propres à EMS et aux plugins.

Formulaires packages.

Vues packages.

Pages
Compositions de vues.

Activables (menu), possibilité de limiter à certains rôles utilisateurs.

Thèmes
CSS des éléments

Thèmes, installation, activation, gestion, mise à disposition des utilisateurs.

Administration

Utilisateurs et rôles

Paramètres
Paramètres obligatoires :

- Nom de l'application [`app_name`] : utilisé sur les pages utilisateurs, par défaut : « EMS ».
- Thèmes :
  - Thème de l'administration [`theme_admin`] (`CONCEPTOR` et +),
  - Thème des utilisateurs [`theme_user`] (`DOMAINE_USER` et +).

Utilisateurs

Affichage des pages

Barre d'outils

Extensions

Notifications

Raccourcis

Profil utilisateur

Déconnexion

Page d'accueil utilisateur

Communication

Messages

Groupes

Projets

Organisation

Planning

Tâche du jour

Rapports

Contraintes techniques

Contraintes de codage

Respect des normes du FIG

Le projet suit les normes :

- PSR-12 pour le format du code source,
- PSR-4 pour le chargement automatique de classes.

Normes propres au framework

Les noms des classes, variables, paramètres sont en anglais.

- Propriété de classe (attribut private et public get/set) ou paramètre de fonction : « `nomDePropriété` »,
- Variable locale : « `local_variable_name` »,
- Propriété de classe privée : « `_private_class_property` ».

Les textes sont en français. Il est prévu qu'ils soient multilingues.

Conventions proposées aux développeurs-utilisateurs du framework

- `VendorName/PackageName`
- `VendorName/PLUGIN_NAME`
- `view-name.html.twig`
- `ActionClass::page_ActionName()`
- `ActionClass::action_ActionName()`
- EMS Routes (url = « `EMS/{admin, user, domain}/route-name` ») :
  `« route-name » ; « HTTP_METHOD » ; « ActionClass » ; « ActionName »`
  Groupes de routes (protégé `@security.yml`) :
  - `AdminRoutes`
  - `UserRoutes`
  - `Routes du domaine (en BDD)`

Gestionnaire de version et branches

Le projet EMS est géré par GIT.

Branches principales

- `master` : branche qui contient la copie de la dernière correction (`x.x.x`),
- `dev` : branche qui contient la copie de la dernière version (`x.x-dev`) mergée officiellement `dev` → `master`

Branches de développement

- `0.1-dev` : branche pré `1.0-dev` → `dev`
- `x.x-dev` : branche de développement d'une version `x.x-dev` → `x.x` / `dev`
- `x.x` : branche principale créée pour la release → `master`
- `feat_x.x_ftName` : branche de développement d'une fonctionnalité → `x.x-dev`
- `bugf_x.x_ftName` : branche de développement d'un bugfix → `x.x-dev`
- `hotf_x.x_ftName` : branche de développement d'un hotfix → `x.x` / `dev`

Tags

`x.x.x`

Environnement de développement

- Docker Ubuntu 20.04
  - Apache 2.4
  - PHP 7.4
  - MySQL server

Référence EMS

Framework EMS

Modification du menu d'administration

…

Récupération des données du package principal

…

Packages EMS

- `EMS/Client`
- `EMS/Product`
- `EMS/Command` (Quote → Command → Order) utilise `EMS/AUTO_MAILER` pour envoyer des courriels aux clients et `EMS/Client`, `EMS/Product`.

Extensions EMS

- `EMS/AUTO_MAILER` : envoie un courriel en paramètre à l'email en paramètre lorsque l'action en paramètre est exécutée (`ActionClass::ActionName`).
  - Config [`from`] : valeur par défaut : `dev@ems.io`
  - `EMS/AUTO_MAILER:sendEmail($title, $emailHtml, $emailTo, $action)`

Créer un package exportable

Exemple gestion Client (EMS/Client).

…

Créer une extension

`Extension::before(doAction, beforeAction)`, `Extension::after(doAction, afterAction)`, mis en place par la méthode `Extension::setup()`;

…

Créer un thème

…